Software Testing and Quality Assurance
Module 3: Testing Metrics

Measurement objectives

Test execution can be tracked effectively using software metrics. Software metrics play a vital role in understanding, controlling, and improving the test process. The organizations that can control their software testing processes are able to predict costs and schedules and increase the effectiveness, efficiency, and profitability of their business.

The objectives for assessing a test process should be well-defined. A number of metrics programs begin by measuring what is convenient or easy to measure, rather than by measuring what is needed. Such programs often fail because the resulting data are not useful to anyone. A measurement program will be more successful, if it is designed with the goals of the project in mind. For this purpose, we can take the help of a goal question metric (GQM):

The GQM approach is based on the fact that the objectives of measurement should be clear, much before the data collection begins. According to this approach, the objectives of a measurement process should be identified first. The GQM approach, with reference to test process measurement, provides the following framework:

- Lists the major goals of the test process.
- Derives from each goal, the questions that must be answered to determine if the goals are being met.
- Decides what must be measured in order to answer the questions adequately.

In this way, we generate only those measures that are related to the goals. In many cases, several measurements may be required to answer a single question. Likewise, a single measurement may apply to more than one question. Software measurement is effective only when the metrics are used and analysed in conjunction with one another.

Category	Attributes to be Measured
Progress	• Scope of testing • Test progress • Defect backlog • Staff productivity • Suspension criteria • Exit criteria
Cost	• Testing cost estimation • Duration of testing • Resource requirements • Training needs of testing group and tool requirement • Cost-effectiveness of automated tool
Quality	• Effectiveness of test cases • Effectiveness of smoke tests • Quality of test plan • Test completeness
Size	• Estimation of test cases • Number of regression tests • Tests to automate

Attributes and corresponding metrics

An organization needs to have a reference set of measurable attributes and corresponding metrics that can be applied at various stages during the course of execution of an organization-wide testing strategy. The attributes to be measured depend on the following factors:

- Time and phase in the software testing lifecycle.
- New business needs.
- Ultimate goal of the project.

The following attributes are defined under the **progress** category of metrics:

1. **Scope of testing.** It helps in estimating the overall amount of work involved, by documenting which parts of the software are to be tested, thereby estimating the overall testing effort required. It also helps in identifying the testing types that are to be used for covering the features under test.

2. **Tracking test progress.** The progress of testing should also be measured to keep in line with the schedule, budget, and resources. If we monitor the progress with these metrics by comparing the actual work done with the planning, then the corrective measures can be taken in advance, thereby controlling the project.

For measuring the test progress, well-planned test plans are prepared wherein testing milestones are planned. Each milestone is scheduled for completion during a certain time period in the test plan. These milestones can be used by the test manager to monitor how the testing efforts are progressing. For example, executing all planned unit tests is one example of a testing milestone. Measurements need to be available for comparing the planned and actual progress towards achieving the testing milestones. To execute the testing milestones of all planned unit tests, the data related to the number of planned unit tests currently available and the number of executed unit tests on this date should be available.

The testing activity being monitored can be projected using graphs that show the trends over a selected period of time. The test progress S curve compares the test progress with the plan to indicate corrective actions, in case the testing activity is falling behind schedule. The advantage of this curve is that schedule slippage is difficult to ignore. The test progress S curve shows the following set of information on one graph:

- Planned number of test cases to be completed successfully by a week.
- Number of test cases attempted in a week.
- Number of test cases completed successfully by a week.

3. **Defect backlog.** It is the number of defects that are outstanding and unresolved at one point of time with respect to the number of defects occurring. The defect backlog metric is to be measured for each release of a software product for release-to-release comparisons. One way of tracking it is to use defect removal percentage for a prior release so that decisions regarding additional testing can be wisely taken in the current release. If the backlog increases, the bugs should be prioritized according to their criticality.

4. **Staff productivity.** Measurement of testing staff productivity helps to improve the contributions made by the testing professionals towards a quality testing process. For example, as test cases are developed, it is interesting to measure the productivity of the staff in developing these test cases. This estimation is useful for the managers to estimate the cost and duration for testing activities. The basic and useful measures of a tester's productivity are:
    - Time spent in test planning
    - Time spent in test case design.
    - Number of test cases developed.
    - Number of test cases developed per unit time.

5. **Suspension criteria.** These are the metrics that establish conditions to suspend testing. It describes the circumstances under which testing would stop temporarily. The suspension criteria of testing describes in advance that the occurrence of certain events/conditions will cause the testing to stop temporarily. With the help of suspension metrics, we can save precious testing time by raising the conditions that suspend testing. Moreover, it indicates the shortcomings of the application and the environmental setup. The following conditions indicate the suspension of testing:
    - There are incomplete tasks on the critical path.
    - Large volumes of bugs are occurring in the project.
    - Critical bugs.
    - Incomplete test environments (including resource shortages).

6. **Exit criteria.** The exit criteria are established for all the levels of a test plan. It indicates the conditions that move the testing activities forward from one level to the next. If we are clear when to exit from one level, the next level can start. On the other hand, if the integration testing exit criteria are not strict, then system testing may start before the completion of integration testing. The exit criteria determine the termination of testing effort which must be communicated to the development team prior to the approval of the test plan.

The following attributes are defined under the **cost** category of metrics:

1. **Testing cost estimation.** The metrics supporting the budget estimation of testing need to be established early. It also includes the cost of test planning itself.

2. **Duration of testing.** There is also a need to estimate the testing schedule during test planning. As a part of the testing schedule, the time required to develop a test plan is also estimated. A testing schedule contains the timelines of all testing milestones. The testing milestones are the major testing activities carried out according to a particular level of testing. A work break-down structure is used to divide the testing efforts into tasks and activities. The timelines of activities in a testing schedule matches the time allocated for testing in the overall project plan.

3. **Resource requirements**. Test planning activities have to estimate the number of testers required for all testing activities with their assigned duties planned in the test plan.

4. **Training needs of testing groups and tool requirements.** Since the test planning also identifies the training needs of testing groups and their tool requirements, we need to have metrics for training and tool requirements.

5. **Cost-effectiveness of automated tools.** When a tool is selected to be used for testing, it is beneficial to evaluate its cost-effectiveness. The cost-effectiveness is measured by taking into account the cost of tool evaluation, tool training, tool acquisition, and tool update and maintenance.

The following attributes are defined under the **quality** category of metrics:

- Effectiveness of test cases. The test cases produced should be effective so that they uncover more and more bugs. The measurement of their effectiveness starts from the test case specifications. The specifications must be verified at the end of the test design phase for conformance with the requirements. While verifying the specifications, emphasis must be given to those factors that affect the effectiveness of test cases including the test cases with incomplete functional specifications, poor test designs, and wrong interpretation of test specifications by the testers.
- Common methods available for verifying test case specifications include inspections and traceability of test case specifications to functional specifications. These methods seldom help to improve the fault-detecting ability of test case specifications.

There are several measures based on faults to check the **effectiveness** of test cases. Some of them are discussed here:

a) Number of faults found in testing.
b) Number of failures observed by the customer which can be used as a reflection of the effectiveness of test cases.
c) Defect-removal efficiency is another powerful metric for test-effectiveness, which is defined as the ratio of the number of faults actually found in testing and the number of faults that could have been found in testing. There are potential issues that must be taken into account while measuring the defect-removal efficiency. For example, the severity of bugs and an estimate of time by which the customers would have discovered most of the failures are to be established. This metric is more helpful in establishing the test effectiveness in the long run as compared to the current project.
d) Defect age is another metric that can be used to measure the test effectiveness, which assigns a numerical value to the fault, depending on the phase in which it is discovered. Defect age is used in another metric called defect spoilage to measure the effectiveness of defect-removal activities. Defect spoilage is calculated as:

$$\text{Spoilage} = \frac{\text{Sum of (Number of defects} \times \text{Defect age)}}{\text{Total number of defects}}$$

Spoilage is more suitable for measuring the long-term trend of test-effectiveness. Generally, low values of defect spoilage mean more effective defect discovery processes. The effectiveness of a test case can also be judged on the basis of the coverage provided. It is a powerful metric in the sense that it is not dependent on the quality of software and also, it is an in-process metric that is applicable when the testing is actually done.

Consider a project with the following distribution of data and calculate its defect spoilage.

SDLC phase	No. of defects	Defect age
Requirement Specs.	34	2
HLD	25	4
LLD	17	5
Coding	10	6

Solution

$$\text{Spoilage} = (34 \times 2 + 25 \times 4 + 17 \times 5 + 10 \times 6)/86 = 3.64$$

Effectiveness of smoke tests. Smoke tests are required to ensure that the application is stable enough for testing, thereby assuring that the system has all the functionality and is working under normal conditions. This testing does not identify faults, but establishes confidence over the stability of a system. The tests that are included in smoke testing cover the basic operations that are most frequently used, e.g. logging in, addition, and deletion of records. Smoke tests need to be a subset of the regression testing suite. It is time-consuming and human-intensive to run the smoke tests manually, each time a new build is received for testing. Automated smoke tests are executed against the software, each time the build is received, ensuring that the major functionality is working.

Quality of test plan. The quality of the test plan produced is also a candidate attribute to be measured, as it may be useful in comparing different plans for different products, noting down the changes observed, and improving the test plans for the future. The test plan should also be effective in giving a high number of errors. Thus, the quality of a test plan is measured in concern with the probable number of errors.

To evaluate a test plan, Berger describes a multi-dimensional qualitative method using rubrics. Rubrics take the form of a table, where each row represents a particular dimension and each column represents a ranking of the dimension. There are ten dimensions that contribute to the philosophy of test planning. These ten dimensions are:

- Theory of objective
- Theory of scope
- Theory of coverage
- Theory of risk
- Theory of data
- Theory of originality
- Theory of communication
- Theory of usefulness
- Theory of completeness
- Theory of insightfulness

Measuring test completeness. This attribute refers to how much of code and requirements are covered by the test set. The advantages of measuring test coverage are that it provides the ability to design new test cases and improve existing ones. There are two issues:
(i) whether the test cases cover all possible output states and
(ii) the adequate number of test cases to achieve test coverage. The relationship between code coverage and the number of test cases is described by the following expression:

$$C(x) = 1 - e^{-(p/N) * x}$$

where C(x) is the coverage after executing x number of test cases, N is the number of blocks in the program, and p is the average number of blocks covered by a test case during the function test. The function indicates that test cases cover some blocks of code more than others, while increasing test coverage beyond a threshold is not cost-effective.

Test coverage for control flow and data flow can be measured using spanning sets of entities in a program flow graph. A spanning set is a minimum subset of the set of entities of the program flow graph such that a test suite covering the entities in this subset is guaranteed to cover every entity in the set.

At the system testing level, we should measure whether all the features of the product are being tested or not. Common requirements coverage metric is the percentage of requirements covered by at least one test. A requirements traceability matrix can be used for this purpose.

The following attributes are defined under the size category of metrics:
- Estimation of test cases. To fully exercise a system and to estimate its resources, an initial estimate of the number of test cases is required. Therefore, metrics estimating the required number of test cases should be developed.
- Number of regression tests. Regression testing is performed on a modified program that establishes confidence that the changes and fixes against reported faults are correct and have not affected the unchanged portions of the program.

However, the number of test cases in regression testing becomes too large to test. Therefore, careful measures are required to select the test cases effectively. Some of the measurements to monitor regression testing are:

- Number of test cases re-used
- Number of test cases added to the tool repository or test database
- Number of test cases rerun when changes are made to the software
- Number of planned regression tests executed
- Number of planned regression tests executed and passed

Tests to automate. Tasks that are repetitive in nature and tedious to perform manually are prime candidates for an automated tool. The categories of tests that come under repetitive tasks are:

- Regression tests
- Smoke tests
- Load tests
- Performance tests

If automation tools have a direct impact on the project, it must be measured. There must be benefits attached with automation including speed, efficiency, accuracy and precision, resource reduction, and repetitiveness. All these factors should be measured.

Estimation models for estimating test efforts

Halstead metrics

Halstead developed expressions for program volume V and program level PL which can be used to estimate testing efforts. The program volume describes the number of volume of information in bits required to specify a program. The program level is a measure of software complexity. Using these definitions, Halstead effort e can be computed as:

PL = 1/ [(n1 / 2) * (N2 / n2)]
e = V/PL
Percentage of testing effort (k) = e(k) / ∑e(i)
Where,

- V is program volume
- PL is Program level
- e(k)=effort for module k
- ∑e(i)=effort across all module of the system

In a module implementation of a project, there are 17 unique operators, 12 unique operands, 45 total operators, and 32 total operands appearing in the module implementation. Calculate its Halstead effort.

Solution

$$n = n1 + n2 = 17 + 12 = 92$$
$$N = N1 + N2 = 45 + 32 = 77$$
$$V = N \log_2 n = 77 \times \log_2 92 = 502.3175$$
$$\text{PL} = 1/[(n1/2) \times (N2/n2)] = 3/68 = 0.044$$
$$e = V/PL = 11416.30$$

Development Ratio method

Another method of estimating tester-to-developer ratios, based on heuristics, is proposed by K. Iberle and S. Bartlett. This method selects a baseline project(s), gathers testers-to-developers ratios, and collects data on various effects like developer-efficiency at removing defects before testing, developer-efficiency at inserting defects, defects found per person, and the value of defects found. After that, an initial estimate is made to calculate the number of testers based upon the ratio of the baseline project. The initial estimate is adjusted using professional experience to show how the above-mentioned effects affect the current project and the baseline project.

Project-staff ratio method

The project-staff ratio method makes use of historical metrics by calculating the percentage of testing personnel from the overall allocated resources planned for the project. The percentage of a test team size may differ according to the type of project.

Project type	Total number of project staff	Test team size %	Number of testers
Embedded system	100	23	23
Application development	100	8	8

Test procedure method:

	Number of test procedures (NTP)	Number of person-hours consumed for testing (PH)	Number of hours per test procedure = PH/ NTP	Total period in which testing is to be done (TP)	Number of testers = PH/TP
Historical Average Record	840	6000	7.14	10 months (1600 hrs)	3.7
New Project Estimate	1000	7140	7.14	1856 hrs	3.8

This model is based on the number of test procedures planned. The number of test procedures decides the number of testers required and the testing time. Thus, the baseline of estimation here is the quantity of test procedures. But you have to do some preparation before actually estimating the resources. It includes developing a historical record of projects including the data related to size (e.g. number of function points, number of test procedures used) and test effort measured in terms of personnel hours. Based on the estimates of historical development size, the number of test procedures required for the new project is estimated.

The only thing to be careful about in this model is that the projects to be compared should be similar in terms of nature, technology, required expertise, and problems solved. The template for this model can be seen in Table 11.4. For a historical project, person-hours consumed for testing test procedures were observed and a factor in the form of number hours per test procedure was calculated. This factor is used for a new project where the number of test procedures are given. Using this factor, we will calculate the number of person-hours for the new project. Then, with the knowledge of the total expended period for this new project, we are able to calculate the number of testers required.

Task planning method

In this model, the baseline for estimation is the historical records of the number of personnel hours expended to perform testing tasks. The historical records collect data related to the work break-down structure and the time required for each task so that the records match the testing tasks. First, the number of person-hours for the full project is calculated as in the test procedure method.

Number of Test Procedures (NTP)	Person-hours consumed for testing (PH)	Hours per test procedure = PH/NTP
840	6000	7.14
1000 (New project)	7140	7.14

The historical data is observed to see how much time has been consumed on individual test activities. After estimating the average idea on historical data, the total person-hours are distributed, which is later adjusted according to some conditions in the new project.

Testing activity	Historical value	% time of the project consumed on the test activity	Preliminary estimate of person hours	Adjusted estimate of person-hours
Test planning	210	3.5	249	
Test design	150	2.5	178	
Test execution	180	3	214	
Project total	6000	100%	7140	6900

Next, the test team size is calculated using the total adjusted estimate of person-hours for the project.

	NTP	PH (Adjusted estimate)	Number of hours per test procedure = PH/NTP	TP	Number of testers = PH/TP
New project estimate	1000	6900	6.9	1856 hrs	3.7

Architectural Design Metrics

1. Structural complexity: It is defined as

$$S(m) = f^2_{out}(m),$$

where S is the structural complexity and $f_{out}(m)$ is the fan-out of module m. This metric gives us the number of stubs required for unit testing of the module m. Thus it can be used in unit testing.

2. Data complexity: This metric measures the complexity in the internal interface for a module m and is defined as

$$D(m) = v(m) / [f_{out}(m) + 1]$$

where v(m) is the number of input and output variables that are passed to and from module m. This metric indicates the probability of errors in module m. As the data complexity increases, the probability of errors in module m also increases. For example, module X has 20 input parameters, 30 internal data items, and 20 output parameters. Similarly, module Y has 10 input parameters, 20 internal data items, and 5 output parameters. Then, the data complexity of module X is more as compared to Y, therefore X is more prone to errors. Therefore, testers should be careful while testing module X.

3. System complexity: It is defined as the sum of structural and data complexity:

$$SC(m) = S(m) + D(m)$$

Since the testing effort of a module is directly proportional to its system complexity, it will be difficult to unit test a module with higher system complexity. Similarly, the overall architectural complexity of the system (which is the sum total of system complexities of all the modules) increases with the increase in each module's complexity. Consequently, the efforts required for integration testing increase with the architectural complexity of the system.

Information Flow Metrics:

Researchers have used information flow metrics between modules. For understanding the measurement, let us understand

the way the data moves through a system:

- Local direct flow exists if
  - a module invokes a second module and passes information to it.
  - the invoked module returns a result to the caller.
- Local indirect flow exists if the invoked module returns information that is subsequently passed to a second invoked module.
- Global flow exists if information flows from one module to another via a global data structure.

The two particular attributes of the information flow can be described as follows:

- Fan-in of a module m is the number of local flows that terminate at m, plus the number of data structures from which information is retrieved by m.
- Fan-out of a module m is the number of local flows that emanate from m, plus the number of data structures that are updated by m.

Henry and Kaufra Design Metric

Henry and Kafura's information flow metric is a well-known approach for measuring the total level of information flow between individual modules and the rest of the system. They measure the information flow complexity as:

$$IFC(m) = length(m) \times ((fan\text{-}in(m) \times fan\text{-}out(m))^2$$

Higher the IF complexity of m, greater is the effort in integration and integration testing, thereby increasing the probability of errors in the module.

Cyclomatic Complexity Measures: McCabe's cyclomatic complexity can be used in the following ways:

1. Since the cyclomatic number measures the number of linearly independent paths through flow graphs, it can be used as the set of minimum number of test cases. If the number of test cases is lesser than the cyclomatic number, then you have to search for the missing test cases. In this way, it becomes a thumb rule that a cyclomatic number provides the minimum number of test cases for effective branch coverage.

2. McCabe has suggested that ideally, the cyclomatic number should be less than or equal to 10. This number provides a quantitative measure of testing difficulty. If the cyclomatic number is more than 10, then the testing effort increases due to the following reasons:
   a. The number of errors increases.
   b. Time required to detect and correct the errors increases.

Thus, the cyclomatic number is an important indication of the ultimate reliability. The amount of test design and test effort is better approximated by the cyclomatic number as compared to the lines of code (LOC) measure. A project in the USA that applied these rules did achieve zero defects for at least 12 months after its delivery.

Function Point Metrics

The function point (FP) metric is used effectively for measuring the size of a software system. Function-based metrics can be used as a predictor for the overall testing effort. Various project-level characteristics (e.g. testing effort and time, errors uncovered, number of test cases produced) of past projects can be collected and correlated with the number of FP produced by a project team. The team can then project the expected values of these characteristics for the current project.

Listed below are a few FP measures:

1. Number of hours required for testing per FP.
2. Number of FPs tested per person-month.
3. Total cost of testing per FP.
4. Defect density measures the number of defects identified across one or more phases of the development project lifecycle and compares that value with the total size of the system. It can be used to compare the density levels across different lifecycle phases or across different development efforts. It is calculated as: number of defects (by phase or in total) / total number of FPs.
5. Test case coverage measures the number of test cases that are necessary to adequately support thorough testing of

a development project. This measure does not indicate the effectiveness of test cases, nor does it guarantee that all conditions have been tested. However, it can be an effective comparative measure to forecast anticipated requirements for testing that may be required on a development system of a particular size. This measure is calculated as: Number of test cases / Total number of FPs

Capers Jones estimates that the number of test cases in a system can be determined by the function points estimate for the corresponding effort. The formula is:

- Number of test cases = $(\text{function points})^{1.2}$

Function points can also be used to measure the acceptance test cases. The formula is:

Number of test cases = (function points) × 1.2

In a project, the estimated function points are 760. Calculate the total number of test cases in the system and the number of test cases in acceptance testing. Also, calculate the defect density (number of total defects is 456) and test case coverage.

Solution

$$\text{Total number of test cases} = (760)^{1.2} = 2864$$

$$\text{Number of test cases for acceptance testing} = (760) \times 1.2 = 912$$

$$\text{Defect density} = 456/760 = 0.6$$

$$\text{Test case coverage} = 2864/760 = 3.768$$

The above relationships show that test cases grow at a faster rate than function points. This is intuitive because, as an application grows, the number of inter-relationships within the applications becomes more complex. For example, if a development application has 1,000 function points, there should be approximately 4,000 total test cases and 1,200 acceptance test cases.

Test Point Analysis

Test point analysis is a technique to measure the black-box test effort estimation. The estimation is for system and acceptance testing. The technique uses function point analysis in the estimation. TPA calculates the test effort estimation in test points for highly important functions, according to the user and also for the whole system. As the test points of the functions are measured, the tester can test the important functionalities first, thereby predicting the risk in testing. This technique also considers the quality characteristics, such as functionality, security, usability, efficiency, etc. with proper weightings.

Procedure for Calculating TPA

For TPA calculation, first the dynamic and static test points are calculated Dynamic test points are the number of test points which are based on dynamic measurable quality characteristics of functions in the system. Dynamic test points are calculated for every function. To calculate a dynamic function point, we need the following:

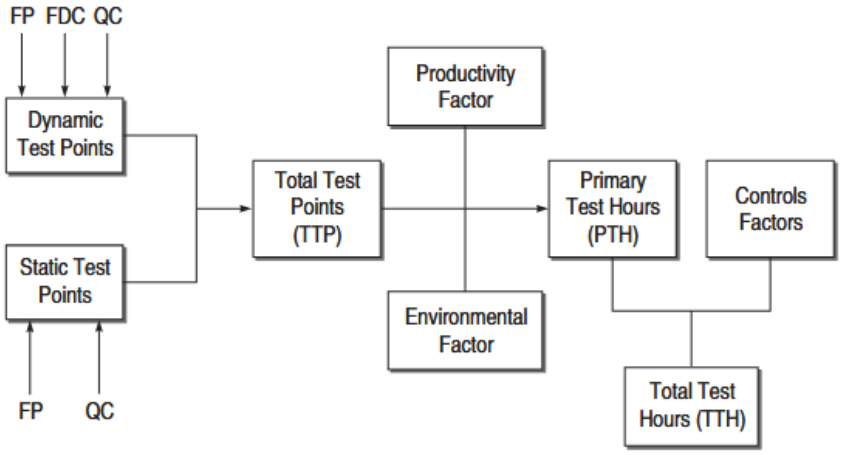
- Function points (FPs) assigned to the function.
- Function dependent factors (FDC), such as complexity, interfacing, function-importance, etc.
- Quality characteristics (QC).

The dynamic test points for individual functions are added and the resulting number is the dynamic test point for the system. Similarly, static test points are the number of test points which are based on static quality characteristics of the system. It is calculated based on the following:

- Function points (FPs) assigned to the system.
- Quality requirements or test strategy for static quality characteristics (QC).

After this, the dynamic test point is added to the static test point to get the total test points (TTP) for the system. This total test point is used for calculating primary test hours (PTH). PTH is the effort estimation for primary testing activities, such as preparation, specification, and execution. PTH is calculated based on the environmental factors and productivity factors. Environmental factors include development environment, testing environment, testing tools,etc. Productivity factor is the measure of experience, knowledge, and skills of the testing team.

Secondary testing activities include management activities like controlling the testing activities. Total test hours (TTH) is calculated by adding some allowances to secondary activities and PTH. Thus, TTH is the final effort estimation for the testing activities.



Calculating Dynamic Test Points

The number of dynamic test points for each function in the system is calculated as:

$$DTP = FP \text{ X } FDC_w \text{ x } QC_{dw}$$

where
DTP = number of dynamic test points
FP = function point assigned to function
FDC_w = weight-assigned function-dependent factors
QC_{dw} = quality characteristic factor, wherein weights are assigned to dynamic quality characteristics

FDC_w is calculated as

$$FDC_w = ((FI_w + UIN_w + I + C) /20) \text{ x } U$$

where
FI_w = function importance rated by users
UIN_w = weights given to usage intensity of the function, i.e. how frequently the function is being used
I = weights given to the function for interfacing with other functions, i.e. if there is a change in function, how many functions in the system will be affected
C = weights given to the complexity of function, i.e. how many conditions are in the algorithm of function
U = uniformity factor

The ratings done for providing weights to each factor of FDC_w :

Factor/ Rating	Function Importance (FI)	Function usage Intensity (UIN)	Interfacing (I)	Complexity (C)	Uniformity Factor
Low	3	2	2	3	0.6 for the function, wherein test specifications are largely re-used such as in clone function or dummy function. Otherwise, it is 1.
Normal	6	4	4	6	
High	12	12	8	12	

QC_{dw} is calculated based on four dynamic quality characteristics, namely suitability, security, usability, and efficiency. First, the rating to every quality characteristic is given and then, a weight to every QC is provided. Based on the rating and weights, QC_{dw} is calculated.

$C_{dw} = \sum$ (rating of QC / 4) x weight factor of QC.
Rating and weights are provided based on the following characteristics:

Characteristic/ Rating	Not Important (0)	Relatively Unimportant (3)	Medium Importance (4)	Very Important (5)	Extremely Important (6)
Suitability	0.75	0.75	0.75	0.75	0.75
Security	0.05	0.05	0.05	0.05	0.05
Usability	0.10	0.10	0.10	0.10	0.10
Efficiency	0.10	0.10	0.10	0.10	0.10

Calculating Static Test Points

The number of static test points for the system is calculated as:

$$STP = FP \times \sum QC_{sw} / 500$$

where

STP = static test point

FP = total function point assigned to the system

QC_{sw} = quality characteristic factor, wherein weights are assigned to static quality characteristics

Static quality characteristic refers to what can be tested with a checklist. QC_{sw} is assigned the value 16 for each quality characteristic which can be tested statically using the checklist.

$$\text{Total test points (TTP)} = DTP + STP$$

Calculating Primary Test Hours

The total test points calculated above can be used to derive the total testing time in hours for performing testing activities. This testing time is called primary test hours (PTH), as it is an estimate for primary testing activities like preparation, specification, execution, etc. This is calculated as:

$$PTH = TTP \times \text{productivity factor} \times \text{environmental factor}$$

- Productivity factor. It is an indication of the number of test hours for one test point. It is measured with factors like experience, knowledge, and skill set of the testing team. Its value ranges between 0.7 to 2.0. But explicit weightage for testing team experience, knowledge, and skills have not been considered in this metric.
- Environmental factor. Primary test hours also depend on many environmental factors of the project. Depending on these factors, it is calculated as:
  Environmental factor = weights of (test tools + development testing + test basis + development environment + testing environment + testware) / 21

Test tools. It indicates the use of automated test tools in the system. Test tool ratings:

1	Highly automated test tools are used.
2	Normal automated test tools are used.
4	No test tools are used.

Development testing. It indicates the earlier efforts made on development testing before system testing or acceptance testing for which the estimate is being done. If the development test has been done thoroughly, then there will be less effort and time needed for system and acceptance testing, otherwise, it will increase.

2	Development test plan is available and test team is aware about the test cases and their results.
4	Development test plan is available.
8	No development test plan is available.

Test basis. It indicates the quality of test documentation being used in the system.

3	Verification as well as validation documentation are available.
6	Validation documentation is available.
12	Documentation is not developed according to standards.

Development environment. It indicates the development platforms, such as operating systems, languages, etc.

2	Development using recent platform.
4	Development using recent and old platform.
8	Development using old platform.

Test environment. It indicates whether the test platform is a new one or has been used many times on the systems.

1	Test platform has been used many times.
2	Test platform is new but similar to others already in use.
4	Test platform is new.

Testware. It indicates how much testware is available in the system.

1	Testware is available along with detailed test cases.
2	Testware is available without test cases.
4	No testware is available.

Calculating Total Test Hours

Primary test hours is the estimation of primary testing activities. If we also include test planning and control activities, then we can calculate the total test hours.

Total test hours = PTH + Planning and control allowance

Planning and Control allowance is calculated based on the following factors:

Team size

3	≤ 4 team members
6	5–10 team members
12	>10 team members

Planning and control tools

2	Both planning and controlling tools are available.
4	Planning tools are available.
8	No management tools are available.

Planning and control allowance (%)
= weights of (team size + planning and control tools)

Planning and control allowance (hours)
= planning and control allowance (%) × PTH

Thus, the total test hours is the total time taken for the whole system. TTH can also be distributed for various test phases, as given in Table 11.16.

Testing phase	% of TTH
Plan	10%
Specification	40%
Execution	45%
Completion	5%

Calculate the total test points for a module whose specifications are: functions points = 414, ratings for all FDC_w factors are normal, uniformity factor =1, rating for all QC_{dw} are 'very important' and for QC_{sw}, three static qualities are considered.

Solution

$$FDC_w = ((FI_w + UIN_w + I + C)/20) \times U = ((6+4+4+6)/20) \times 1 = 1$$

$$QC_{dw} = \Sigma \text{ (rating of QC / 4)} \times \text{weight factor of QC}$$
$$= (5/4 \times 0.75) + (5/4 \times 0.05) + (5/4 \times 0.10) + (5/4 \times 0.10)$$
$$= 0.9375 + 0.0625 + 0.125 + 0.125 = 1.25$$

$$DTP = FP \times FDC_w \times QC_{dw}$$
$$= 414 \times 1 \times 1.25 = 517.5$$

$$STP = FP \times \Sigma QC_{sw} / 500$$
$$= 414 \times ((16 \times 3)/500) = 39.744$$

$$\text{Total test points (TTP)} = DTP + STP = 517.5 + 39.744 = 557.244$$

Testing Progress Metrics

Everyone in the testing team wants to know when the testing should stop. To know when the testing is complete, we need to track the execution of testing. This is achieved by collecting data or metrics, showing the progress of testing. Using these progress metrics, the release date of the project can be determined. These metrics are collected iteratively during the stages of the test execution cycle.

Test Procedure Execution Status: It is defined as:

Test Proc Exec. Status (%) = Number of executed test cases / Total number of test cases

This metric ascertains the number of percentage of test cases remaining to be executed.

Defect Aging: Defect aging is the turnaround time for a defect to be corrected. This metric is defined as:

Defect aging = Closing date of bug – Start date when bug was opened

This metric data for various bugs is used for a trend analysis such that it can be known that n number of defects per day can be fixed by the test team. For example, suppose 100 defects are being recorded for a project. If documented, the past experience indicates that the team can fix as many as 20 defects per day, the turnaround time for 100 problem reports may be estimated at one work-week.

Defect Fix Time to Retest: It is defined as:

Defect fix time to retest = Date of fixing the bug and releasing in new build – Date of retesting the bug

This metric provides a measure that the test team is retesting all the modifications corresponding to bugs at an adequate rate. If the rate of fixing and retesting them is not on time, then the project progress will suffer and consequently will increase the cost of the project.

Defect Trend Analysis: defined as the trend in the number of defects found as the testing life cycle progresses. For eg:

- Number of defects of each type detected in unit test per hour
- Number of defects of each type detected in integration test per hour

To strengthen the defect information, defects of a type can also be classified according to their severity of impact on the system. It will be useful for testing, if we get many defects of high severity. For example,

- Number of defects over severity level X (where X is an integer value for measuring the severity levels)

The trend can be such that the number of bugs increases or decreases as the system approaches its completion. Ideally, the trend should be such that the number of newly opened defects should decrease as the system testing phase ends. If the trend shows that the number of bugs are increasing with every stage of testing:

- Previous bugs have not been closed/fixed properly.
- Testing techniques for testing coverage is not adequate for earlier builds.
- Inability to execute some tests until some of the defects have been fixed, allowing execution of those tests, which then find new defects.

Recurrence Ratio

This metric indicates the quality of bug-fixes. The quality of bug-fixes is good if it does not introduce any new bug in the previous working functionality of the software and the bug does not re-occur. But if the bug-fixing introduces new bugs in the already working software or the same bug re-occurs, then the quality is not good. Moreover, this metric indicates to the test team the degree to which the previously working functionality is being adversely affected by the bug-fixes. When the degree is high, the developers are informed about it. This metric is measured as:

Number of bugs remaining per fix

Defect Density: This metric is defined as:

$$Defect\ density\ =\ \frac{Total\ number\ of\ defects\ found\ for\ a\ requirement}{Number\ of\ test\ cases\ executed\ for\ that\ requirement}$$

It indicates the average number of bugs found per test case in a specific functional area or requirement. If the defect density is high in a specific functionality, then there is some problem in the functionality due to the following reasons:

- Complex functionality
- Problem with design or implementation of that functionality
- Inadequate resources are assigned to the functionality

This metric can also be utilized after the release of the product. The defect data should also be collected after the release of the product. For example, we can use the following metric:

Pre-ship defect density / Post-ship defect density

This metric gives the indication of how many defects remain in the software when it is released. Ideally, the number of pre-release defects found should increase and post-release defects should decrease.

Coverage Measures: Coverage goals can be planned, against which the actual ones can be measured. For white-box testing, a combination of following is recommended: degree of statement, branch, data flow, basis path, etc. coverage (planned, actual). In this way, testers can use the following metric:
Actual degree of coverage / Planned degree of coverage

Similarly, for black-box testing, the following measures can be determined:
(i) number of requirements or features to be tested, and
(ii) number of equivalence classes identified.

In this way, testers can use the following metric:

- Number of features or equivalence classes actually covered / Total number of features or equivalence classes.

These metrics will help in identifying the work to be done.

Tester Productivity Measures

Testers' productivity in unit time can also be measured and tracked. Managers can use these metrics for their team's

productivity and build a confidence level for the on-date delivery of the product. The following are some useful metrics in measuring the testers' productivity:

- Time spent in test planning
- Time spent in test case design
- Time spent in test execution
- Time spent in test reporting
- Number of test cases developed
- Number of test cases executed

Each of these metrics is planned and actual ones are measured and used for tracking productivity.

Budget and Resource Monitoring Measures

The budget and other resources involved in a project are well-planned and must be tracked with the actual ones. Otherwise, a project may stop in between due to over-budget or fall behind its schedule.

Earned value tracking is the measure which is used to monitor the use of resources in testing. Test planners first estimate the total number of hours or cost involved in testing. Each testing activity is then assigned a value based on the estimated percentage of the total time or budget. It provides a relative value to each testing activity with respect to the entire testing effort. Then, this metric can be measured for the actual one and compared with the planned earned value.

For the planned earned values, we need the following measurement data:

- Total estimated time or cost for overall testing effort.
- Estimated time or cost for each testing activity.
- Actual time or cost of each testing activity.

Then, the following metric is used to track the cost and time:

Estimated cost or time for testing activity / Actual cost or time of testing activity:

Planned				Actual		
Testing Activity	Estimated Time (HRS)	Estimated Earned Value	Cumulative Earned Value	Date	Actual Earned value	Cumulative Earned Value

This table has two partitions: planned values and actual values. Each testing activity as well as their estimated hours for completion should be listed. The total hours for all the tasks are determined and the estimated earned value for each activity is then calculated based on their estimated percentage of the total time. This gives a relative value to each testing activity with respect to the entire testing effort. The estimated earned values are accumulated in the next column. When the testing effort is in progress, the date and actual earned value for each activity as well as their actual accumulated earned values.

Test Case Effectiveness Metric

It shows how to determine whether a set of test cases is sufficiently effective in revealing defects. It is defined as:

$$TCE = \frac{Number\ of\ defects\ found\ by\ the\ test\ cases}{Total\ number\ of\ defects} X\ 100$$

In fact, a baseline value is selected for the TCE and is assigned for the project. When the TCE value is at or above the baseline, then the test cases have been found effective for testing. Otherwise, the test cases must be redesigned to increase the effectiveness.